S-3 Bucket / Launching Static Website

Module – 4 (Cloud architecture

Create a Bucket -> Give unique name -> ACL enable -> Disclick Block all public access (Uncheck) -> I acknowledge -> Bucket version – Enable -> Create bucket.

=> terminal

gitclone cafestatic to Downloads and change permission

=> sudo ~~chmod~~ chown -R msis:msis .

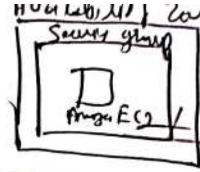
Now you can able to upload the files.

L) Upload all files of cafestatic

-> go to created S3 bucket -> Upload -> files & folders -> -> Upload -> Go to AWS console, S3 -> choose bucket -> properties -> static website hosting -> Edit -> Enable -> Index document – name of the index.html (cafestatic website name) -> Save changes

=> go to bucket -> Select your bucket -> Select all objects -> Actions -> Make public using ACL -> Make Public -> After these go to properties -> static website hosting -> copy URL and Paste Page will open// now you can see URL

EC-2



Launching Dynamic Website on EC-2

Cloud Foundations Module - 6 - Lab - 3

Create EC2 instance, -> Enter name -> Ubuntu -> -> Ubuntu 24.04 -> t2 micro -> key pair - vockey -> Network Setting Edit -> VPC - Lab(VPC) -> Public Subnet 1, -> enable Auto - assign -> Create security group -> Provide security group name (CafeDbSg) -> Inbound security group rules -> SSH -> Anywhere / My IP, http -> Anywhere, https -> Anywhere -> Configure storage -> gp3 -> Launch instance

~~Download Dynamic Website using git clone~~ -> Download Pem key -> cd Downloads -> chmod 600 Pemkey (labsuser.pem)

ssh -i Pemkey (labsuser.pem) ubuntu@IPaddress (Public IP of ubuntu)

```
sudo apt-get update -y
sudo apt-get install apache2 php libapache2-mod-php
        php-mysql mysql-server mysql-client -y
git clone 'github repo'
sudo systemctl status apache2
cd /var/www/html
ls
sudo rm -rf *
```

~~git clone 'github repo'~~

```
sudo mv CafeDynamic-Website/mompocafe/* .
ls
sudo nano /etc/apache2/sites-available/000-default.conf
        set to /var/www/html
```

optional {
```
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html
```

```
sudo systemctl restart apache2
```

__/__/__

sudo mysql -u root -p

<mysql> Create database msiscafedb;
CREATE USER 'myuser'@'localhost' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON msiscafedb.* TO 'myuser'@'localhost';

<mysql> exit

cd CafeDynamic_Website/mompopdb — Inside this directory there is create-db.sql
sudo mysql -u myuser -p

<mysql> show database;
use msiscafedb;
Show tables;
Source create-db.sql;
Show tables;
Select * from product_group;

<mysql> exit
cd ..
cd
cd var/www/html
sudo nano getAppParameters.php
< set everything >

change -> user name
url
db name
password

=> Sudo systemctl restart apache2

Now you can see dynamic website

(EC-2)

Launching Dynamic Website on EC-2

Cloud Foundations Module - 6 - Lab - 3

Create EC2 instance, -> Enter name -> Ubuntu ->
-> Ubuntu 24.04 -> t2 micro -> Key pair - vockey -> Network setting Edit -> VPC - Lab(VPC) -> Public Subnet 1 -> enable Auto - assign ->
Create security group -> Provide security group name (Cafedbsg)
-> Inbound security group rules -> SSH -> ~~Anywhere~~ My IP
HTTP -> Anywhere
HTTPS -> Anywhere
-> Configure Storage -> gp3 -> Launch instance

~~Download Dynamic Website using git clone~~ -> Download Pem key -> cd Downloads
-> chmod 600 Pemkey (labsuser.pem)
ssh -i Pemkey (labsuser.pem) ubuntu@IPaddress (Public IP of Ubuntu)

sudo apt-get update -y
sudo apt-get install apache2 php libapache2-mod-php
php-mysql mysql-server mysql-client -y
git clone 'github repo'
sudo systemctl status apache2
cd /var/www/html
ls
sudo rm -rf *
~~git clone 'github repo'~~
sudo mv CafeDynamic_Website/mompocafe/* .
ls
sudo nano /etc/apache2/sites_available/000-default.conf
set to /var/www/html

optional {
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html

sudo systemctl restart apache2

_/_/_

sudo mysql -u root -p

<mysql> Create database msiscafedb;
CREATE USER 'myuser'@'localhost' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON msiscafedb.* TO 'myuser'@'localhost';

<mysql> exit

cd CafeDynamic_Website/mompopdb
sudo mysql -u myuser -p

Inside this directory there is create-db-sql

<mysql> show database;
use msiscafedb;
Show tables;
Source create-db.sql;
Show tables;
Select * from product_group;

<mysql> exit
cd ..
cd ..
CD var/www/html
sudo nano getAppParameters.php
< set everything >

change -> user name
URL
db name
password

=> sudo systemctl restart apache2

Now you can see dynamic website

VPC — Virtual Private cloud

CIDR (Classless Inter-Domain Routing)
NAT (Network Address Translation)
private ip to Public ip

Module-5 → Lab-2 Build your VPC and launch a Web server

to 170.16.51.0/24 → 170.16.51.255/24 — $32-24 = 8$, $2^8 = 255$ addresses; last 8 bits are used to allocate

to 172.16.0.0/16 → $\underbrace{172.16}_{\text{Network Identification}}.\underbrace{255.255}_{\text{Host Identification}}/\underbrace{16}_{\text{Subnet mask}}$ — $32-16=16$, $2^{16}=$ addresses; last 16 bits are used to allocate

1 Subnet → 1 Route Table
1 Route Table → Many subnets

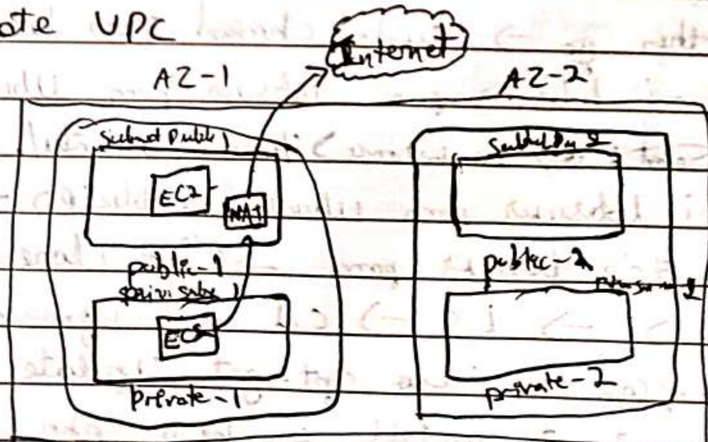
Public & Private instances within a VPC can speak to eachother

NAT gateway does 3 things:-
1] allows private instance to each internet
2] Converts private ip to Public ip
3] blocks internet traffic from coming in

Sudo scp -i labusers.pem labuser.pem ubuntu@IP:/home/ubuntu

Start

⇒ Create VPC



VPC — Virtual Private Cloud

Start

=> Create VPC → VPC & more → Name - Cafe Web → 2 Public & 2 Private → No of Availability Zone 2 → Customize subnets CIDR blocks →
Auto generated

east 1a → Public 10.0.0.0/24 | east 1b - 10.0.2.0/24

private - east 1a - 10.0.1.0/24 | east 1b → 10.0.3.0/24 → NAT gateway → In 1 AZ →

VPC endpoints → None → Create VPC → View VPC

Public Instance :-

=> EC2 Search → Create EC2 → give name → Ubuntu → 24.04 → t2 micro → Vockey → Network Settings

Edit VPC - CafeWeb → Public subnet 1a → Public IP
Auto assign public IP → Enable → Security group - Web sg → Inbound Rules

SSH → My IP | http - Anywhere | HTTPS - Anywhere

→ gp3 → launch

Private instance :-

=> Same steps to create EC2 but difference is Network Settings → Private | east 1a (Subnet)

Disable - Public IP → Security group name → ~~Cafe web sg~~ dbsg

it should be different from Public → Inbound rules

SSH - ~~My IP~~ Anywhere → MySql/Aurora - Custom → Source security group - Cafe sg → Launch instance

Download Pem → AWS Details → Download go to terminal it will be there now

terminal => Check it in Downloads then ~~go~~ → sudo chmod 700 labsuser.pem

→ scp -i labsuser.pem labsuser.pem ubuntu@<Public IP of ~~1a east~~ 1a instance>:/home/ubuntu/. →

ssh -i labsuser.pem ubuntu@<Public IP> → ls →

chmod 700 labsuser.pem → sudo git clone <url from git of cafe-dynamic> → ls → cd Cafe-Dynamic → cd /mompopcafe → sudo apt-get update -y →

→ sudo apt-get install apache2 php libapache2-mod-php php-mysql mysql-client -y →

__/__/____

-> cd cafe-Dynamic -> cd mompopcafe -> ~~sudo~~ ls -> sudo cp -rf *(snack) /var/www/html/. -> sudo rm -rf /var/www/html/index.html -> sudo systemctl restart apache2 -> copy public ip & paste in URL it shows front page -> then -> cd -> ls -> ~~from~~ their labsuser.pem is present from their -> ssh -i labsuser.pem ubuntu@<private ip> -> sudo apt-get update -y -> sudo ~~apt-ge~~ apt-get install ~~apache2~~ mysql-server -> sudo mysql -u root -p -> Create database cafedb;
-> show databases; -> Create user 'msis'@'%.' identified by 'msis@123'; -> ~~grant all privileges on 'msis'@'%'~~ grant all privileges on cafedb.* to 'msis'@'%.'; -> exit (from mysql)
=> sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
-> bind address = 0.0.0.0 -> sudo systemctl restart mysql -> exit -> Public Ubuntu now
cd cafe-Dynamic -> cd mompopcafe
-> sudo mysql -h <private ip> -u msis -p
-> mysql will come -> show databases; -> use cafedb;
-> source create-db.sql; -> show tables;
-> exit -> ~~sudo systemctl restart~~
cd /var/www/html -> sudo nano getAppParameters
-> URL -> Private IP -> name -> cafedb -> user -> msis
password - msis@123 save & exit -> sudo system restart apache2 -> then go to page refresh
it check database it show Now.

__/__/__

IAM (Identity Access Management) :-

1] IAM group - Created for a group of people that requires similar access.

2] IAM user - Created for a single user.

3] IAM Role - Created for services to access another Service.

4] Identity Federation - Created for users of an application, They can sign in using other accounts like facebook, gmail, etc.

IAM Policies are of 2 types :- 1] AWS managed (aws created)
2] custom inline (user created)

use JSON editor to create a JSON policy and attach that to your user, group or role.

When we add a user to a group, they will inherit all the policies associated with that group.

Wednesday

Lambda

17/09/25

* Serverless Architecture
  - Scheduling of task.
  - Milli second of code running.
  - manage only code & Libraries -> could not exceed 250 MB.

Lambda function -> Running of your code (only when triggered).

Cloud Foundation
Module - 6 -> Activity - AWS Lambda
Compute

Lab => to just stop ~~and start~~ a task (EC2 instance - a)

Event Bridge Event -> Lambda function triggered (Python code) -> EC2 instance Stoped (Stop)

~~use~~ two services
search EC2 -> open in new tab -> Search lambda function -> open in new tab
-> in EC2 2 instance are running
-> lambda function -> Create ~~instance~~ function ->
a) Author from scratch -> name - mylambda -> Runtime (platform) - Python 3.11 -> Architecture - X86_64 (intel) -> Change default execution role -> use an existing role -> Existing role - mystopinator-Role -> Create function

scroll Down - their is code -
- Now go to lab workbench copy code it is in 13th point copy & paste in lambda code (Replace old code with copied one)

-> Replace • region = 'us-east-1', and
instances = [~~paste~~ [copy instance ID from EC2] & paste here]
write in single cotes ' '
- to get this go to EC2 select instance scroll down then instance ID is there

in leftside -> Deploy

~~Lambda/lambda~~

Now create a trigger

In ~~lamb~~ lambda only

add trigger -> Event Bridge (cloud watch Events)
-> create a new rule -> Rule name ~~-~~ my rule
-> Rule description - optional -> Rule type -
schedule expression - rate (1 minute)
-> add

-> ~~Go for test~~ in lambda -> Refresh the Page of EC2 Now
Now it stops the instance

How to start it automatically work on it?
↳ it won't start because in this Lab we don't have permission to start it.

Lambda

SNS

17/09/25

AWS Architecting

Module 14) guided lab: Implementing Serverless Architecture on AWS

in directory from ip in URL : 172.18.181.60:8000
download items.csv & store.csv

Start Lab. 1–

Search -> lambda open in new tab -> DynamoDB, S3 bucket
in new tab -> SNS in new tab

Lambda -> Create function -> Name - Loadinventorylambda
-> Python 3.9 -> x86_64 -> Permission
-> use an existing role - lambda-load-inventory-Role -> Create function

go to web directory -> 172.18.181.60:8000
- inventory-store-lambda.txt copy
from import json, urllib, boto3, csv
paste in lambda code (or)
Deploy it
go to guided lab & copy the # load inventory lambda function

-> S3 :–

configure to send a Event

Create bucket -> Name - Inventory-189 (unique) not same
-> Create bucket.
-> Click of inventory-189 -> properties ->
Event notification -> Create event notification
-> inventoryload (Name) -> Event type
-> ☑ All object Create event -> scroll down
-> Destination -> ◉ Lambda function ->
Choose from your lambda function ->
select -> loadinventorylambda
Save changes

-/-/-

→ Bucket → inventory-189. click on it → Upload a file → store.csv Upload it

Uploaded

~~Now → Dynamo DB → Select Inventory D → Run it~~

Now
go to dashboard (Main Page) → Copy Dashboard link (AWS Details) →
open in new tab → it shows Inventory Dashboard now.

Now go to lambda create another one lambda function

⇒ Create function → Name → storenotification → python 3.9
→ x86_64 → permissions → use an existing role → lambda-check-stock-Role → Create function

go to Directory 172.18.181.60:800 → inventory_store_lambda → copy stock
(or) check lambda function code and paste it in lambda code → Deploy it.

take it from guided lab & paste here # stock check lambda function

SNS Simple Notification Service
Create topic → it should be like that below in code it is like that only
Name: NoStock → Next Step →
Display name - Cafeinventory → Create topic

⇒ Creat Subscription → Protocol - Email → Endpoint -

Create Subscription

__/__/__

Now go to your mail and click on confirm subscription

Now to trigger it go to S3

=) Upload -) items.csv -) Upload

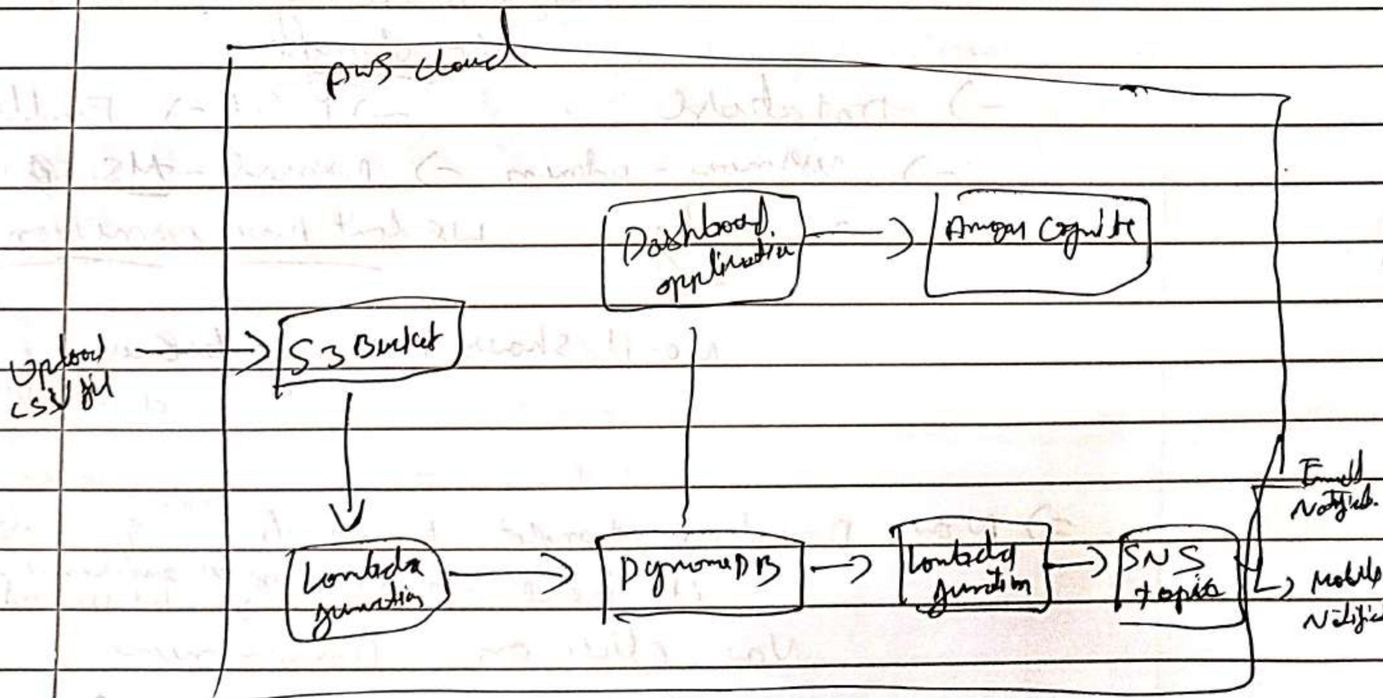
Now to trigger it then go to SNS & run it by selecting the inventory

=) Lambda -) store notification -)

=) add trigger - DynamoDB -) DynamoDB table -) choose inventory (arn: aws - - -)

add

Now we have to get the Notification in mail



Platform as a Servic

Cloud Foundation -> Module-6 -> Activity: AWS Elastic Beanstalk

Load balancer

To create
1) Automatically Scale up & Scale Down
2) Highly Available, Fault-tollerance
3) Elastic Infrastructure

AWS Elastic Beanstalk do all this by it own

Lab Search -> Elastic Beanstalk -> click on Environment name -> if you click on Domain -> no app is running it tell Not Found.

-> go back
-> we need java application to run.
-> click on configuration (in left side) -> Instance traffic & Scaling -> Edit -> No change.
-> Database -> Edit -> Enable it
-> username - admin -> Password - MSis@12345
-> apply   we don't have permission

No it shows warning becouse we created a database so.

=> Now Download template from web 172.18.181.60:8000 & deploy it here - it deploy go to environment -> open it -> select upload & deploy

Now click on Domain name now you can open the page.

__/__/__

to creat our own Environment

Create Environment -> Web Server -> name - cgfgrp-env
-> platform - PHP -> Next it throws an error
because we don have permission

Thursday

18|09|25

2 types of Scaling

Theory Part

① EC2 auto Scaling ② Auto Scaling

Elastic Load Balancer :-

types of load balancer :-

① Application load balancer. => work at Application level (7 layer of OSI)

② gateway —||— (older) (previous generation)

③ classic version load balancer.

④ Network load balancer. -> Routes a traffic to the target group on the IP protocol.

cloud watch :- is a monitoring & observability of services that gives you a insight into your aws.

Auto Scaling & monitoring
Load Balancer & Auto Scaling group

Thursday

18/04/25

cloud Foundation -> Module - 10

Lab - 6 : Scale and Load Balance your architecture

click on EC2 -> instances -> Select Web Server 1 ->
copy Public ip -> paste in Browser type
http://<ip>

(1) => Now to create a image of server
=> select instance web server -> Actions ->
~~create~~ image & template -> Create Image ->
name - my app -> Tag -> Tag - Name -> Value -
myappname -> Create image

(2) => Creating Load balancer
=> click on Load Balancer left side
-> Create a Load balancer -> Application Load
Balancer -> Create -> name - myelb ->
internet-facing ✓ -> VPC -> (Lab VPC) ✓
-> Availability zone & subnets
select ☑ Public Subnet 1 for us east 1a
-> ☑ US - east-1 b -> Public Subnet 2

=> Security group - remove default -> Select ->
web - Security group -> Create target group ☐ -> click it

(3) => create target group (select) -> name ->
myec2 targetgroup -> Next -> ~~created~~ Next
Create target group

(4) => goto load balancer page Now
refresh it Now select the
target group myec2targetgroup
-> Create Load Balancer

__/__/__

Creating a Autoscaling group

(5) => Click on Launch template left side -> Create launch template -> Name - myasgtemplate -> ☑ provide guidance to help me set up a template - - - - Autoscaling -> Application & OS Images -> click on My AMIS -> ☑ owned by me -> AMI - Myapp -> Instance type - t2.micro -> key pair - Vockey -> Security groups - Websecurity groups -> -> Advanced details -> ~~[illegible]~~ -> scroll down -> Enable - detail monitoring cloudwatch -> Create launch template ✓

VPC subnet ✗

(6) => go to Launch template -> Select it -> Actions -> Create a autoscaling group -> name - myasg -> Next -> Step 2 -> VPC - Lab VPC -> -> Availability zones & subnets -> US - east - 1a (Private 1) -> US - east - 1 b (Private Subnet 2) select both => Next -> step 3 -> attach to an existing load balancer (select) -> select target group -> myecatarget-group | HTTP -> Health check -> ☑ -> Turn on Elastic Load Balancing health check -> Next -> Step 4 -> -> Desired capacity - 2 -> Min desired capacity - 2 -> Max desired capacity - 4 -> Automatic scaling -> Select -> target tracking scaling policy -> instance warmup - 60 seconds -> Additional setting -> Enable group metrics collection within cloudwatch -> Next -> step 5 -> -> Next -> Step 6 -> add tag -> Name -> value optional = mywebserver -> Next -> Step 7 -> Create Auto Scaling group ✓

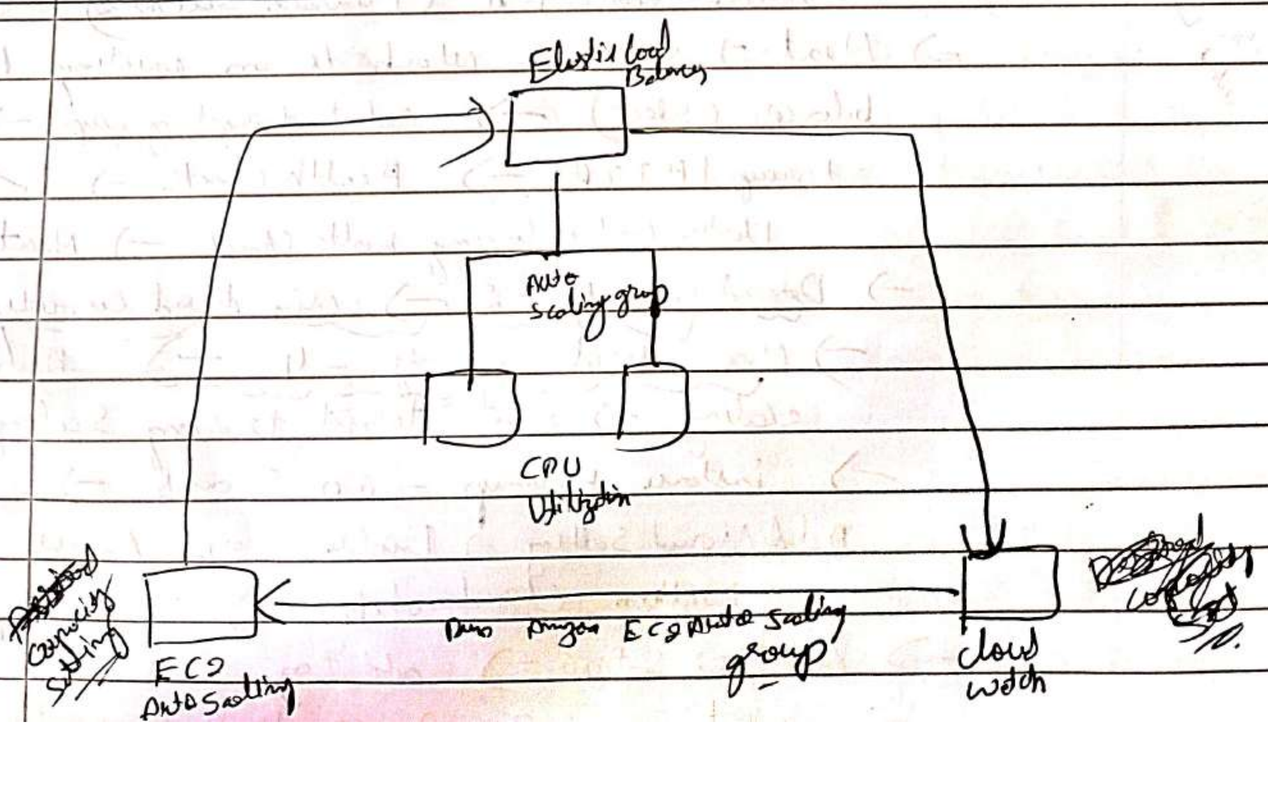
VPC subnet ✓

_/_/_

Remove that running $\boxed{X}$

goto EC2 -> ~~stop the Running~~ -> and now ~~go to~~ refresh you have to see extra 2 more mywebservers -> Now go to -> load balancer -> select it -> click go inside -> copy DNS name and paste in Web now you can see the Page if you refresh it again & again it changes to 1a & 1b alternately depending on Load.

=> cloud watch -> Search cloudwatch open in new tab -> left side alarm created click on it -> in alarm -> it show in alarm -> up to that generate the load in the Web -> refresh (by clicking on load test) the screen after its comes to in-alarm stay go to EC2 refresh it -> now you can see one more Instance here.



Architecture
Lab: (5)

(Amazon EFS) Elastic Filesystem

Compute layer using Amazon EC2
Guided Lab: introducing Amazon Elastic File System (Amazon EFS)
open
① EC2 instance.
② EFS in new tab.

EC2 → EFS client select → copy security group ID at below → go to security group left side → Create new → name - efs mount target sg → Description → efsmounttargetsg → VPC - Lab (VPC) → Add Inbound rule - Type - NFS → custom - paste copied security group ID here → Create it

Done

④ Go to EFS → Create file → Select customize → Name - myefs → regional (✓) → disable the Encryption ☐ Enable Encryption of data at rest → & uncheck automatic backups →
Tag → Name - Name → Tag Value - myefs → Next → VPC → Lab (VPC) →
Security group remove → add efs mount targets for both security group of us east 1a & 1b → Next → Next → Create
error will come ignore it

④ Select myefs → View details → Attach →
there the command is there to mount copy 2nd one → ~~download~~ new ~~key~~ Now in AWS details link is there copy & paste in web session manager will be opened now mount it
→ ~~change permission of key~~
~~Sudo chmod 600 labsuser.pem~~

__/__/____

=> ~~sudo ssh -i labuser.pem ec2-user@~~ (public ip of EC2)

in AWS details there, is one link copy and paste it terminal
Will going to open (Session manager)

sudo mkdir /mnt/efs

copy that attach command from EFS and paste here
with path /mnt/efs (at end of link)
initially before mentioning /mnt/efs their will be other
path at end so delete it & mount this here

=> df -h

=> Now to see the performance go to guided lab 3 0 point & copy paste it to System manager terminal

Architecture (Monolithic Node js)

AWS

CFA

Cloud foundations & Architecting

AWS

30/08/25

RDS

Architecting → Module - 6

RDS

* Unmanaged services:
  ↳ An error occurs you are responsible (like in VPC)
  ↳ Example of IaaS.
* managed services.
  ↳ An error occurs AWS is responsible it will take care of it (RDS) (PaaS)
  ↳ RDS is example of ~~RDS~~ managed services (or) PaaS.

* Session handling → without ssh we can access

Lab

- Search EC2 → Instance → cafeserver → CP public address → Paste in web → add http://44.211.165.184/cafe/ → we can see the cafe web
  (Public IP)
- Search system manager (directly login to browser (ssh)) (open in new tab by right click on it) → left side session manager → start session → select ~~cafedb~~ cafeserver → start session → sudo su (typeit) Enter → su ec2-user → ~~pwd → cd var/www/html/ → ls~~ →

Note: SSM - system manager agent should be installed to have cafedb in (Session manager)

cd cafe → ls

→ ~~cd cafe → ls~~ →,

No need to do this

No need just to do this go to next page

get to Secrets manager in main web → /cafe/dbUrl → retrieve secret value → go back → /cafe/dbPasswords → copy password → go to that root terminal → cd /var/www/html → sudo mysql -h (/cafe/dbURL) ~~and root~~ -P (end point is this in secret manager) → copied password → show databases; → use cafe-db; → Now to use

Start → ⇒ → go to services (search RDS) → Aurora RDS → Databases (open it in new tab)
to copy data & populate it we do this

→ create database → Standard create → Aurora ~~(5 times faster than mysql)~~ → but choose mysql → Production ~~(to update 4 all we use it)~~ → ~~sandbox dev/test~~ sandbox → Single - AZ DB

→ Provide identifier (cafedb) (It can be anything) → to create username & Password self managed → user: admin → Password: MSIS1934 → Burstable classes (t classes) → t micro → storage → gp2 → storage - 20 →

→ ~~Connectivity (default keep it)~~ → VPC → lab VPC → P.T.O

_ /_ /_

VPC firewall choose dbsg ✓ remove default → uncheck Enhanced monitoring → Click Create database → (End point created)

~~take the End Point address from cafe db that you were created in RDS~~

→ VPC Security group → Security group id → Next click

Inbound rule → Edit inbound rules → Type → MySQL Aurora → Source custom → Select (Cafe sg) → Save

(to migrate database to RDS)

→ go to Session manager → terminal → mysql

[ec2-user@cafe server cafe] $ mysql -h (Paste End Point) -u admin -p → Enter password MSIS1234 → MySQL →

→ Show ~~tables~~; databases; → Create database cafe-db; → Show databases; → exit → CD → ~~Now in [ec2-user@cafe server cafe] $~~ →

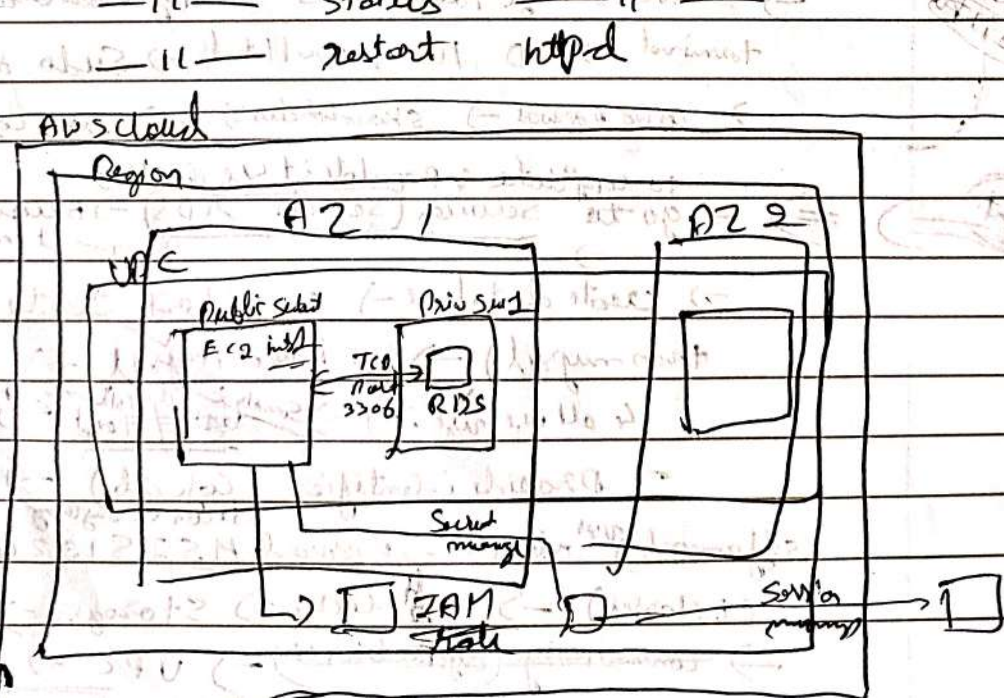
[ec2-user@cafeserver] mysqldump --databases cafe-db -u root -p > cafe.sql → Paste Password (Restart ...) →

→ nano cafe.sql (ctrl+x) → mysql -h (cafedb.c7...... End point url) -u admin -p → MSIS1234 → Show databases; →

→ ~~source cafe.sql~~ → Show tables; → use cafe-db; → ~~show tables~~; → source cafe.sql; → Show tables

⇒ change username & Password, & URL in Secret manager

- sudo systemctl stop mariadb.service
- sudo — " — status — " —
- sudo — " — restart httpd



AMAZON EC2 — AMI

04/09/25

Thursday

⊛ How to quickly launch new Web Server with cafe webapp
⊛ to quickly take a Snap shot

=> How to increase the Size of hard DISK:-

Introduction to
=> cloud foundation - Module (6) compute -> Amazon EC2 Lab3 ->
(We have shell Script in 172.18.181.60:8000 / amazon-linux-ec2 ...) -> we have shell script here!!

=> Go inside Lab -> EC2 -> Launch Instance -> My cafewebServer
-> AWS (Amazon linux) -> AMI -> Amazon Linux 2023 kernel
-6.1 AMI => t2.micro -> Vockey -> Edit
-> VPC - (LabVPC) -> Enable (Public subnet 1 public subnet) -> Security group Name -
Websg -> SSh - Anywhere -> add Rule -> HTTP (to specific eg you)
- Anywhere => Advance detail -> IAM instance
- leave it as it is don't select => Termination Protection
gp3 - Enable => Stop protection - Enable => don't do / leave it as it is don't Select

1) Private IP
2) Public IP
└> not static
3) Elastic IP address
└> static
└> per region 5 is the limit

it is in URL amazon linux ec2
=> user data => copy shell script from URL => & Paste
it has which start from #! / bin / bash -
Note => It only execute only one time (first time execution) (first time system booted)
we can also configure it.

=> (Launch instance)

(To get Screenshot) -> Select -> cafewebservice static -> Actions ->
Monitor & troubleshoot -> get instance Screenshot
(Wait) for few seconds to get active
=> Copy the public IP & Paste in New tab Cafe Page
will going to open.

Note: Service Quota
There we can launch
many instance

Persistence storage -> if you terminate also we can have the data.

Snapshot -> capturing current state of instance

=> If we go to terminate mycafewebserver it won't get terminated
(Instance state) -> terminate

=> If we go to stop it will stop (because we didn't enabled it in settings)
(After ~~stop~~ it show stop)

=> Actions -> Instance Settings -> change instance type
t2.small -> change instance type.

Virtual CPU
RAM
Networking
Performance

=> go to Storage -> Volumes ~~monitoring~~ -> click on it
Select it -> Actions -> modify the Volume -> change 8 to 10 ->
save it.
(go to instance -> refresh screen) It changed to 8 to 10 (in Volume)

(4 types of EMI)
① AWS start EMI
② AWS market space
③
④

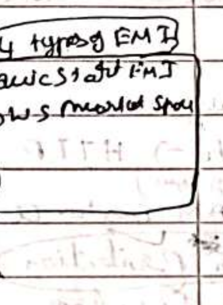
=> select instance -> actions -> Image & template -> Create an image
i.e: taking snapshot of EBS
=> Image name -> My cafe ~~...~~ ami -> Create a image
throws a error Because the user don't have root permission.

EBS

Module-7

Lab-4 Working with EBS (Elastic Block Store)

To do?
-> How do you create Volume
-> & How do you attach a Volume to instance & take a snapshot



Start
=> Search EC2 -> Instance -> Select Lab -> ~~Storage~~ Storage -> ~~Volume~~ ~~Create~~ Volume
~~Create new volume~~
Click on link ~~Click on link~~ -> Create a Volume -> gp3 -> Size - 1
-> Tags -> Key - Name -> Value -> MyVolume
=> Create a volume (capital) => scroll right side
volume should be available now (wait few seconds)

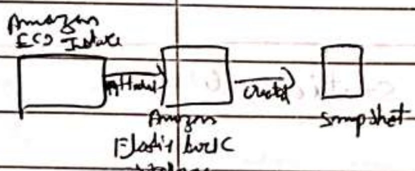
To Attach Volume
-> Select volume ~~Select Volume~~ ~~Select Lab~~ -> Actions -> Attach Volume -> (Lab)
~~Select Lab~~
Instance (Lab) -> /dev/sdf (Device name) -> Attach Volume

Mounting - ~~Creating a~~ file is the process of making a storage resource (like EBS or EFS) accessible to EC2 instance by attaching it to the instance file system so that it can be used just like a local folder or disk.
(ie : the changes made will also reflect to main folder)

Now one volume state goes to available to in-use

Mounting :

=> go to instance -> Select lab -> Select Connect -> SSH client (NO change)
-> EC2 instance connect (No change) -> click Connect -> it ~~show~~ display something
in that type => df -h (type)
(to check this volume exists or not) => sudo file -s /dev/xvdf (nothing)
(Now we create a file) => sudo ~~mkfs~~ mkfs -t ext3 /dev/xvdf
-> used to create Linux type
=> To do mounting
new folder created => sudo mkdir /mnt/myvolume
/dev/xvdf mounted -> /mnt/myvolume | here we are mounting the folder => sudo mount /dev/xvdf /mnt/myvolume
=> df -h



=> cd /mnt/myvolume
=> sudo touch file.txt
=> sudo chown -R ec2-user: ec2-user .
=> cat >> file.txt
write something (Enter) (ctrl + c)
=> cat file.txt

taking snapshot of my volume now
=> go to instance page -> left side go to Volume -> Select volume -> actions -> create a snapshot -> type tag Name & MyVolume snapshot in tags Create a volume.

=> go to snapshot in left side | Refresh page
Select my volume snapshot -> Actions -> Create volume from ~~copy~~ snapshot
-> Add tags -> Name -> value - Restoredvolume
=> Create volume

volume 2 => go to Volume in left side part
=> select Restored volume -> Actions -> Attach Volume
-> Instance - (lab) -> Device name - /dev/sde
=> Attach volume

URL page
localhost:8080 → Settings → nodes → add new node →
<hostname> - <IP>
Ubuntuserver - 52 (any name) → Permanent node → create
→ Number of executors (How many job we can do) → Remote
root directory → /home/msis/jenkins (where we need to create a workspace) → Launch method → launch agent by connecting it to the controller → Save.

* for Every slave node it generate a jar file
* click on the created slave node → if it is not showing the IP instead it showing the localhost means go to settings → system → scroll down & change & save

---

⇒ ssh <slave name>@<slave ip> → sudo mkdir jenkins
⇒ cd jenkins → sudo curl (paste the curl part from jenkins)
→ ~~sudo~~ ls (agent should be there now) → sudo java --
(copy & paste the java line from that node in jenkins)
→ Now it is connected it will now eliminated the not connected thing in the node which we have created.

Create a new in jenkins → <Enter name> → Free Style → create → ☑ restrict where this project →
Ubuntuserver_52 → git, Post URL → Build step → Execute shell → sudo make lint → Save
sudo make lint-fix
sudo make test
sudo make test-report
sudo make run

→ Build → it wont stop

↳ go to URL paste slave ip → info →
there slave ip should be their.

nodejs_monitoring
nodejs - docker_deploy

__/__/__

Node.js

go to Web → 172.18.181.60:800 → Link_To_github
→ nodejs - monitoring - demoapp → open in new page

⇒ Clone it to terminal
⇒ cd to nodejs - - - - → cd src → cd ..
⇒ ~~sudo rm -rf node_modules~~
⇒ sudo docker build . -f build/Dockerfile -t nodemonitoringapp : V1
⇒ sudo docker run --name nodeapp -it -d -p # Run image
3001:3000 nodemonitorapp : V1

Jenkins Pipeline - Now tag created → go to URL typeip
ip: 172.18.181.10:3001

Nodejs
To make pipeline

you will get all this in Jenkins Ubuntuserver_52

Jenkins :- terminal :- (1) ~~sudo~~ ssh . cmsis@172.18.181.52 (Adithya in)
(3) curl -sO http://172.18.181.10:8000/jnlpJars/agent.jar
(4) sudo java -jar agent.jar -URL http://172.18.181.10:8001 -secret ad - - - -

Jenkins ⇒ Create → New item → nodejs_docker_deploy → freestyle
→ create → Restrict where this project can be run - Ubuntuserver_52 (Slave System) (Adithya system) → Post git URL of nodejs monitoring demoapp.git → Build Steps → Execute shell → command

first remove the agent.jar if exists

↳ sudo docker rm -f nodemonitoring || true
↳ sudo docker build . -f build/Dockerfile -t nodemonitorapp: V1 (space)
↳ sudo docker run --name nodemonitoringapp -it -d -p 3002:3000 nodemonitorapp : V1

Save

⇒ then go to nodejs_monitoring_bare → Configure
→ don't restrict it → Put git URL of node monitoring - demoapp → Build Steps →
Execute shell → commands.

_/_/_

↳ sudo apt-get install nodejs npm -y
sudo make lint
sudo make lint-fix
sudo make test
sudo make test-report

⇒ Post-build Actions
↳ add → Trigger parameterized build on other project
→ Projects to build → select → nodejs-docker-deploy
→ Stable → ☑ Trigger build without parameters
Save build

Now create pipeline → Click on +
↳ name → nodemonitoring-CI-CD → Build pipeline
View → Create ✓
pipeline Flow → Based on upstream / downstream relationship
↳ select initial job → nodejs-monitoring-base
Save Run
it will succesfully execute
then go to the new page and type
the Slave ip (Adithya ip) ie
172.18.181.52 : 3002

then you can see the Page Now

Exercise
Python → Pratice it on your own
↳ Here use requirements.txt instead of nodemonitoring

similar to above

Completed ✓

(Simple Storage Service.)

S3 Bucket (we can only host static website here, html (files))

CA, M4 challenge (cafe lab): creating a static website for the cafe.

(S3) -> click on this

create bucket

Bucket name

manju123.com -> Has to be unique ends with .com

☐ -> uncheck the, Block all public access

☑ -> check the, I acknowledge -> create -> click on

cd downloads

-> sudo git clone (cafe-static-website) repo -> cd Down -> i get cafe-static-web but locked -> cd

-> sudo chown -R msis:msis cafe-sta-web // -> owns this files directory.

user group

-> upload (css, images, index.html) goto cafe-static website files

-> enable ACL (inside permission)

☑ I acknow click on bucket owner enforced

ACL = Access Control List

-> enable static website hosting (edit) (inside properties)

-> index.html -> name

-> objects (select all uploaded files)

-> Actions (make public using ACL) (make public)

-> in properties at the end i will get a URL, it will open cafe-static-website.

lab 3

EC2 (~~Challenge lab~~ : Introduction to Amazon EC2)

CF, M6 , EC2 -> instance -> launch instance.

name -> webhg

-> Amazon machine imag (AMI)

select [ubuntu]

AMI version.
24.04

-> Instance type -> t2.micro
-> key pair (login) -> vockey

-> Network setting -> lab VPC
Public subnet 1 or 2.
Auto assign public IP :
(enable) for public subnets.
disable for private sub.

-> create security group, | Inbound security group rule.
name -> webhg . | ssh | HTTP
| my ip | anywhere.

(launch)

Go to work bench -> aws details -> download Permkey [delete the old ones]

terminal.
cd Downloads/
ls.
-> labsuser.pem. (read write) binary 110

Permission -> sudo chmod 600 labsuser.pem

ssh to ec2 instance localhost
-> ssh -i labsuser.pem ubuntu@(publicip) on instance.
(identifying this file) loginuser

ubuntu -> sudo apt-get update -y

ubu -> sudo apt-get install apache2 php libapache2-mod-php php-mysql mysql-client mysql-server -y

ubu -> sudo git clone (cafe dynamic repo)

ls.

cd cafe-dy-web X
cd mompop cafe X

sudo rm -rf /var/www/html/index.html

~~cd cafe-dy-~~

sudo cp -rf * /var/www/html/.
sudo systemctl restart apache2
(paste public ip)

cd cafe-dy-web.
cd mompopdb.
ls.
no memu
i have create-db.sql

sudo mysql or sudo mysql -u root -p

-> create database cafe_db;

-> create user 'msis'@'localhost' identified by 'msis@123';

-> grant all privileges on cafe_bd.* to 'msis@'localhost';

exit

ubuntu:- cafe-dy-web/mompopdb; I sudo mysql -u msis -p
password msis@123

mysql-> show databases;

my. use cafe-db;

show tables;
source create.db.sql; select * from product;
exit

ubunt. c-d-w/mompopdb5 ls.

cd /var/www/html
ls
getAppParameters.php.

sudo nano getAppPar.php

sudo systemctl restart apache2.

menu will run

paste public ip